



[딥러닝 실습 보고서] ResNet-50 vs Vision Transformer(ViT) 성능 비교

실험 일자: 2026년 4월 6일

사용 장비: NVIDIA GeForce RTX 3070 Ti (8GB VRAM)

1. 실험 목적 (Introduction)

- 전통적인 CNN 구조인 **ResNet-50**과 최신 Attention 메커니즘 기반의 ****ViT(Vision Transformer)****의 이미지 분류 성능을 비교함.
- 사전 학습된(Pre-trained) 모델을 활용하여 꽃 데이터(30종)에 대한 미세 조정(Fine-tuning) 효율성을 검증함.
- 통계적 평가지표인 **Weighted F1-Score**를 통해 클래스 불균형 상황에서의 모델 신뢰도를 평가함.

2. 실험 환경 및 데이터 (Environment & Dataset)

- Framework:** PyTorch 2.5.1 / CUDA 12.1
- Dataset:** Flowers30 (총 10,784장)
- Data Split (Seed 42):** * **Train:** 7,548장 (70%)
 - Validation:** 1,617장 (15%)
 - Test:** 1,619장 (15%)
- Hyperparameters:** Epochs=10, Batch Size=32, Optimizer=Adam, Learning Rate=1e-4

3. 모델별 성능 비교 (Results)

항목 (Metrics)	ResNet-50 (CNN)	ViT-Tiny (Transformer)
Model Size (Params)	약 2,350만 개	약 570 만 개 (가벼움)
Best Val F1-Score	0.9060	0.9084
Final Test F1-Score	0.90	0.90
총 학습 속도	8m 29.8s	5m 4.4s

4. 실전 이미지 추론 테스트 (Inference Test)

인터넷에서 수집한 독립된 이미지(In-the-wild)를 통해 모델의 일반화 성능을 확인한 결과입니다.

- 테스트 이미지: 아카시아
- **ResNet-50** 결과: 아카시아 / 확신도 (99.98)%
- **ViT-Tiny** 결과: 아카시아 / 확신도 (99.46)%



[ResNet50 Prediction]
Result: Acacia
Confidence: 99.98%

[ViT-Tiny Prediction]
Result: Acacia
Confidence: 99.46%

- 테스트 이미지: 프리지아
- **ResNet-50** 결과: 프리지아 / 확신도 (100)%
- **ViT-Tiny** 결과: 프리지아 / 확신도 (99.29)%



[ResNet50 Prediction]
Result: Freesia
Confidence: 100.00%

[ViT-Tiny Prediction]
Result: Freesia
Confidence: 99.29%

5. 통계적 고찰 및 결론 (Conclusion)

1. 모델 효율성: ViT-Tiny는 ResNet-50 대비 파라미터 수가 약 1/4 수준임에도 불구하고, (비슷한/준수한) 성능을 기록함. 이는 3070 Ti와 같은 GPU 환경에서 메모리 효율성이 매우 높음을 시사함.
2. 구조적 차이 분석:
 - **ResNet-50**: 합성곱(Convolution) 연산을 통해 이미지의 국소적 특징(질감, 모양)을 잘 포착함.
 - **ViT**: 셀프 어텐션(Self-Attention)을 통해 이미지 전체의 문맥적 관계를 파악함.
3. 최종 결론: 본 데이터셋(Flowers30)에서는 ViT가 가장 우수한 일반화 성능을 보였음. 향후 데이터 증강(Data Augmentation) 기법을 추가한다면 ViT의 성능 향상 폭이 더 클 것으로 기대됨.

5. ViT 코드 뜯어보기

```
class VisualTransformerStepByStep(nn.Module):
```

```
    def __init__(self, img_size=224, patch_size=16, in_chans=3, num_classes=30,
embed_dim=192):
```

```
        super().__init__()
```

```
        # [1] Patch Embedding: 이미지를 조각내고 벡터로 변환
```

```
        # 224x224 이미지를 16x16 조각으로 나누면 총 196개의 패치가 생깁니다.
```

```
        # 이 조각들을 Linear 레이어를 통과시켜 192차원의 벡터로 만듭니다.
```

```
        self.patch_embed = nn.Conv2d(in_chans, embed_dim, kernel_size=patch_size,
stride=patch_size)
```

```
        num_patches = (img_size // patch_size) ** 2
```

```
        # [2] CLS Token: 이미지 전체 정보를 대표하는 '반장' 토큰
```

```
        # 통계에서 '상수항(Intercept)'처럼 전체 데이터의 특징을 수렴시키는 역할을 합니다.
```

```
        self.cls_token = nn.Parameter(torch.zeros(1, 1, embed_dim))
```

```
        # [3] Positional Encoding: 조각들의 위치 정보
```

```
        # Transformer는 순서를 모르기 때문에 "나는 1번 조각이야" 같은 좌표값을
더해줍니다.
```

```
        self.pos_embed = nn.Parameter(torch.zeros(1, num_patches + 1, embed_dim))
```

```
        # [4] Multi-Head Self-Attention (핵심 엔진)
```

```
        # 이미지 조각들끼리 "누가 누구와 관련 있는지" 상관관계를 계산합니다.
```

```
        self.transformer_block = nn.TransformerEncoderLayer(
            d_model=embed_dim, nhead=3, dim_feedforward=embed_dim*4,
            batch_first=True
```

)

[5] MLP Head: 최종 분류기

추출된 특징을 바탕으로 꽃 30종 중 무엇인지 확률(Softmax)을 계산합니다.

self.head = nn.Linear(embed_dim, num_classes)

def forward(self, x):

x shape: [Batch, 3, 224, 224] (이미지 데이터 입력)

1단계: 이미지를 패치로 쪼개고 평평하게 펴기 (Flatten)

결과: [Batch, 196, 192] -> 196개의 단어가 각각 192차원 벡터인 셈

x = self.patch_embed(x).flatten(2).transpose(1, 2)

2단계: 맨 앞에 CLS 토큰 추가 (반장 투입)

결과: [Batch, 197, 192]

cls_token = self.cls_token.expand(x.shape[0], -1, -1)

x = torch.cat((cls_token, x), dim=1)

3단계: 위치 정보 더하기 (좌표값 입력)

x = x + self.pos_embed

4단계: Self-Attention 연산 (상관관계 분석)

조각 A가 조각 B를 얼마나 쳐다보아야(Attention) 하는지 결정합니다.

x = self.transformer_block(x)

5단계: 최종 분류 (CLS 토큰의 결과값만 사용)

맨 앞에 추가했던 '반장' 토큰만 꺼내서 30개 클래스로 분류합니다.

outcome = self.head(x[:, 0])

return outcome